Mortgage Pricer — Test Results

Automated Test Documentation for Model Governance

MKM Research Labs

Version 1.0 — 23-March-2026

David K Kelly

Legal Notice

PROPRIETARY AND CONFIDENTIAL

1 Test Results — Mortgage Pricer (MKM-MP-001)

Tests run on **23 March 2026 at 12:58:50**.

Table 1: Test Results Summary — Mortgage Pricer (MKM-MP-001)

Total Tests	67
Passed	67
Failed	0
Skipped	0
Duration	0.05s

Table 2: Detailed Test Results — Mortgage Pricer

Test Description	Acceptance Criteria	Result	Time
Default ids assigned	`results[0]["mortgage_id"] == "MORTGAGE_0"; results[0]["property_id"] == "PROPERTY_0"`	Pass	0.001s
Error isolation	`len(results) == 3; "error" not in results[0]` (+2 more)	Pass	0.001s
Prices multiple mortgages	`len(results) == 2; results[0]["mortgage_id"] == "M1"` (+1 more)	Pass	0.001s
Custom tax rate	`MortgagePricer(tax_rate=0.40).tax_rate == 0.40`	Pass	0.000s
Default tax rate	`MortgagePricer().tax_rate == 0.20`	Pass	0.000s
Custom tax rate	`spread_high_tax > spread_low_tax`	Pass	0.000s
Floor at 10bps	`spread >= 0.001`	Pass	0.000s
Higher affordability higher spread	`spread_low_income > spread_high_income`	Pass	0.000s
Negative income fallback	`spread == 0.15`	Pass	0.000s
Spread bounded by schedule	`spread >= 0.001; spread < 0.05`	Pass	0.000s
Zero income fallback	`spread == 0.15`	Pass	0.000s
debug=True in the credit spread calculation.	`"credit_spread" in result`	Pass	0.001s
debug=True exercises logging branches without error.	`result["mortgage_value"] > 0`	Pass	0.001s
Zero income with debug=True exercises zero-income debug branch.	`result["credit_spread"] > 0`	Pass	0.001s
flood_risk_category=None with debug=True.	`result["mortgage_value"] > 0`	Pass	0.001s

Test Description	Acceptance Criteria	Result	Time
When interest_rate=0, monthly_rate==0 → loan_amount/n_periods formula.	`result["mortgage_value"] > 0`	Pass	0.001s
Batch passes flood risk	`results[0]["flood_risk_factor"] == 1.40; results[1]["flood_risk_factor"] == 1.00` (+1 more)	Pass	0.001s
Case insensitive	`MortgagePricer.calculate_flood_risk_impact("very high") =...; MortgagePricer.calculate_flood_risk_impact("VERY LOW") ==...` (+1 more)	Pass	0.000s
Flood factor in result	`result["flood_risk_factor"] == 1.20`	Pass	0.001s
Flood risk increases spread	`high["credit_spread"] > low["credit_spread"]; high["flood_risk_factor"] > low["flood_risk_factor"]` (+1 more)	Pass	0.001s
Flood risk multipliers[high-1.4] [category='High', expected=1.4]	`MortgagePricer.calculate_flood_risk_impact(category) == e...`	Pass	0.000s
Flood risk multipliers[low-1.05] [category='Low', expected=1.05]	`MortgagePricer.calculate_flood_risk_impact(category) == e...`	Pass	0.000s
Flood risk multipliers[medium-1.2] [category='Medium', expected=1.2]	`MortgagePricer.calculate_flood_risk_impact(category) == e...`	Pass	0.000s
Flood risk multipliers[very high-1.75] [category='Very High', expected=1.75]	`MortgagePricer.calculate_flood_risk_impact(category) == e...`	Pass	0.000s
Flood risk multipliers[very low-1.0] [category='Very Low', expected=1.0]	`MortgagePricer.calculate_flood_risk_impact(category) == e...`	Pass	0.000s
No flood risk defaults to unity	`result["flood_risk_factor"] == 1.0`	Pass	0.001s
None returns unity	`MortgagePricer.calculate_flood_risk_impact(None) == 1.0`	Pass	0.000s
Unknown category returns unity	`MortgagePricer.calculate_flood_risk_impact("Unknown") == 1.0; MortgagePricer.calculate_flood_risk_impact("Extreme") == 1.0`	Pass	0.000s
Ltv thresholds[0.5-1.0] [ltv=0.5, expected=1.0]	`pricer.calculate_loan_to_value_impact(pv * ltv, pv) == ex...`	Pass	0.000s
Ltv thresholds[0.8-1.0] [ltv=0.8, expected=1.0]	`pricer.calculate_loan_to_value_impact(pv * ltv, pv) == ex...`	Pass	0.000s
Ltv thresholds[0.85-1.1] [ltv=0.85, expected=1.1]	`pricer.calculate_loan_to_value_impact(pv * ltv, pv) == ex...`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Ltv thresholds[0.9-1.1] [ltv=0.9, expected=1.1]	`pricer.calculate_loan_to_value_impact(pv * ltv, pv) == ex...`	Pass	0.000s
Ltv thresholds[0.91-1.3] [ltv=0.91, expected=1.3]	`pricer.calculate_loan_to_value_impact(pv * ltv, pv) == ex...`	Pass	0.000s
Ltv thresholds[0.95-1.3] [ltv=0.95, expected=1.3]	`pricer.calculate_loan_to_value_impact(pv * ltv, pv) == ex...`	Pass	0.000s
Ltv thresholds[0.96-1.5] [ltv=0.96, expected=1.5]	`pricer.calculate_loan_to_value_impact(pv * ltv, pv) == ex...`	Pass	0.000s
Ltv thresholds[1.0-1.5] [ltv=1.0, expected=1.5]	`pricer.calculate_loan_to_value_impact(pv * ltv, pv) == ex...`	Pass	0.000s
Zero property value	`pricer.calculate_loan_to_value_impact(100_000, 0) == 1.5`	Pass	0.000s
Higher rate higher payment	`high > low`	Pass	0.000s
Shorter term higher payment	`short > long`	Pass	0.000s
£200k at 3.5% over 25yr £1,001.25/month.	`995 < payment < 1010`	Pass	0.000s
Zero-rate degenerate case: M = P / n.	`payment == pytest.approx(1000.0)`	Pass	0.000s
Each mortgage has property id	`mapping.get("property_id") is not None`	Pass	0.004s
Processing stats	`stats["successful_mortgages"] == 5; stats["failed_mortgages"] == 0`	Pass	0.004s
Output file exists	`result["file_path"].exists()`	Pass	0.005s
Output file is valid json	`isinstance(data, dict)`	Pass	0.005s
Generate returns expected keys	`"data" in result; "file_path" in result` (+1 more)	Pass	0.003s
Mortgage count matches properties	`len(result["data"]["mortgages"]) == 5`	Pass	0.003s
Mortgage id format	`re.match(r"^MORT-[a-f0-9]{8}$", mid)`	Pass	0.003s
Mortgage ids unique	`len(ids) == len(set(ids))`	Pass	0.002s
All errors returns error	`"error" in calculate_portfolio_metrics(results)`	Pass	0.000s
High risk count	`calculate_portfolio_metrics(results)["high_risk_mortgages...`	Pass	0.000s
Mixed results exclude errors	`metrics["total_mortgages"] == 1; metrics["error_count"] == 1`	Pass	0.001s
Valid results	`metrics["total_mortgages"] == 2; metrics["error_count"] == 0` (+3 more)	Pass	0.001s
Balance converges to zero	`result["outstanding_balance"][-1] < 1.0`	Pass	0.001s
Expected losses non negative	`all(el >= 0 for el in result["expected_losses"])`	Pass	0.001s

Test Description	Acceptance Criteria	Result	Time
Fair value less than par	`result["mortgage_value"] < base_mortgage_params["loan_amo...; result["discount_to_par"] > 0` (+1 more)	Pass	0.001s
Hazard rates positive	`result["hazard_rates"][0] == 0.0; all(result["hazard_rates"][i] > 0 for i in range(1, len(r...`	Pass	0.001s
High ltv increases spread	`high_ltv["credit_spread"] > low_ltv["credit_spread"]; high_ltv["ltv_factor"] > low_ltv["ltv_factor"]`	Pass	0.001s
Higher haircut more loss	`high["pv_losses"] > low["pv_losses"]`	Pass	0.001s
Input validation clamps	`result["mortgage_value"] > 0`	Pass	0.000s
Ltv factor in result	`"ltv_factor" in result; result["ltv_factor"] == 1.0`	Pass	0.001s
Raises on invalid types	—	Pass	0.000s
Recovery haircut zero low ltv	`all(lgd == 0 for lgd in result["lgds"])`	Pass	0.001s
Survival first period less than one	`result["survival_probs"][1] < 1.0`	Pass	0.001s
Survival monotonically decreasing	`surv[i] <= surv[i - 1]`	Pass	0.001s
Positive rate exceeds principal	`cost > 200_000`	Pass	0.000s
Zero rate equals principal	`cost == pytest.approx(100_000.0)`	Pass	0.000s

1.1 Test Document History

Date	Version	Description	Author
05-Mar-2026	1.0	Initial test suite (65 tests)	D. Kelly
07-Mar-2026	1.1	62 test(s) added; 65 test(s) removed (total: 62)	D. Kelly
23-Mar-2026	1.2	5 test(s) added (total: 67)	D. Kelly